

JAWABAN SOAL — SISTEM TEMU KEMBALI INFORMASI (STKI)

Konteks tugas: Bahasa Indonesia, pipeline preprocessing 5 tahap (NLTK & Sastrawi), fokus **TF-IDF + Vector Space Model (VSM)**, dilengkapi Boolean Retrieval, Jaccard, Extractive Summarization, dan MinHash.

1. Tahapan dalam Sistem Temu Kembali Informasi (IR)

STKI bekerja dalam dua fase besar: **indexing** (offline, dilakukan sekali di awal) dan **retrieval** (online, saat ada query).

A. Document Acquisition / Crawling Mengumpulkan koleksi dokumen yang akan dicari (korpus). Pada tugas kelompok kami: ekstraksi teks dari PDF jurnal memakai `pdfplumber`.

B. Text Preprocessing — inilah pipeline 5 tahap di laporan kami: 1. **Case folding** — menyeragamkan ke huruf kecil (`Mahasiswa` → `mahasiswa`). 2. **Cleaning / filtering** — membuang angka, tanda baca, simbol (`re.sub(r'^a-z\s', ' ', text)`). 3. **Tokenizing** — memecah kalimat menjadi token/kata. 4. **Stopword removal** — membuang kata umum tak diskriminatif (`yang, di, dan`) memakai NLTK/Sastrawi. 5. **Stemming** — mereduksi kata berimbuhan ke kata dasar dengan algoritma Nazief & Adriani (`mengakses` → `akses`).

C. Indexing Membangun struktur data temu balik cepat, umumnya **inverted index** (term → posting list dokumen). Pada Boolean Retrieval inilah intinya.

D. Term Weighting (Pembobotan) Memberi bobot tiap term, misalnya **TF-IDF**, supaya term penting/diskriminatif berkontribusi lebih besar daripada term umum.

E. Query Processing Query pengguna diproses dengan **pipeline preprocessing yang sama** dengan dokumen (wajib konsisten), lalu direpresentasikan dalam ruang term yang sama.

F. Matching & Ranking Mencocokkan query dengan dokumen dan mengurutkan berdasar skor relevansi — misalnya **cosine similarity (VSM)**, skor TF-IDF, atau Jaccard. Hasil disajikan terurut (ranked list).

G. Evaluation (Evaluasi) Mengukur kualitas hasil dengan metrik seperti **precision, recall, F-measure, MAP** — lalu sistem disempurnakan (relevance feedback) berdasar hasil evaluasi.

2. Perbedaan dan Hubungan VSM dan TF-IDF

Hubungan: keduanya bukan saingan, melainkan **bertingkat (komponen vs kerangka)**. TF-IDF adalah **skema pembobotan** (memberi nilai pada term), sedangkan VSM adalah **model representasi & pengukuran kemiripan** (kerangka geometris). VSM membutuhkan suatu bobot untuk mengisi komponen vektornya — dan bobot yang paling lazim dipakai adalah TF-IDF. Jadi **TF-IDF "mengisi" vektor di dalam VSM**.

Perbedaan:

Aspek	TF-IDF	VSM
Hakikat	Skema pembobotan term (sebuah angka per term)	Model ruang vektor + metode similarity
Output	Bobot pentingnya term di dokumen	Skor kemiripan query–dokumen
Perhitungan skor query	Menjumlahkan (sum) bobot term query	Cosine similarity (dot product / norma)
Normalisasi panjang dokumen	Tidak (TF-IDF Sum tak dinormalisasi)	Ya, dibagi norma vektor
Bisa berdiri sendiri?	Bisa jadi input metode lain (mis. summarization)	Butuh skema bobot (mis. TF-IDF) untuk vektornya

Bukti dari laporan kami (Bab 4): D4 peringkat 1 di kedua metode, tetapi pada peringkat 2–3 berbeda — TF-IDF menaruh D1 di atas D2, sedangkan VSM membalik (D2 di atas D1) karena cosine menormalisasi panjang dokumen, sehingga D2 yang lebih panjang tidak dirugikan. Inilah perbedaan praktis yang nyata: **TF-IDF Sum bias ke akumulasi, VSM mengukur sudut/proporsi.**

3. Apakah stopwords removal diperlukan? Kapan tidak diperlukan?

Diperlukan ketika (sesuai pembobotan di laporan kami): - Sistem berbasis **bag-of-words / TF-IDF / VSM**. Stopword (**yang, di, dan**) muncul sangat sering tetapi tidak diskriminatif; bila dibiarkan ia memperbesar indeks dan **menyesatkan pembobotan** (term umum mendapat skor tinggi semu). - Korpus besar dan butuh efisiensi penyimpanan/kecepatan. - Pencarian berbasis kata kunci (keyword search).

TIDAK diperlukan / justru merugikan ketika: 1. **Phrase / exact-match search**. Frasa seperti *"to be or not to be"* atau *"sistem dari informasi"* hampir seluruhnya stopwords — membuangnya menghancurkan makna frasa. 2. **Model semantik modern** (BERT, word embeddings, model bahasa). Model ini memanfaatkan konteks; stopwords justru memberi sinyal sintaksis/struktur kalimat, jadi sebaiknya dipertahankan. 3. **Tugas yang bergantung struktur kalimat** — sentiment analysis (kata *not/tidak* sering masuk daftar stopwords padahal membalik makna), question answering, machine translation. 4. **Korpus kecil** di mana setiap kata bisa berkontribusi membedakan dokumen.

Singkatnya: **untuk metode statistik klasik (TF-IDF/VSM/Boolean) seperti tugas kami → perlu**; untuk pencarian frasa eksak atau model semantik kontekstual → tidak perlu / hindari.

4. Pengaruh Overstemming & Understemming (contoh Bahasa Indonesia)

Ya, keduanya **mempengaruhi recall dan precision**, karena stemming menentukan term mana yang dianggap sama.

Overstemming — pemotongan **berlebihan**: dua kata berbeda makna direduksi ke akar yang sama. Akibat: **recall naik tapi precision turun** (dokumen tak relevan ikut terambil).

Contoh ID (algoritma Nazief-Adriani yang kami pakai): - **beruang** (hewan) bisa terpotong menjadi **uang** (prefix **ber-**) → query "uang" salah menemukan dokumen tentang beruang. - **kekurangan** → bila prefix **ke-** + suffix **-an** dipangkas agresif bisa menjadi **kurang**, mencampur makna "defisit/kekurangan" dengan "kurang". - **mengetahui** → bisa over-stem ke **tahu**, mencampur makna "mengetahui" dengan "tahu" (makanan).

Understemming — pemotongan **kurang**: kata-kata yang sebenarnya satu akar tidak disatukan. Akibat: **precision tetap tapi recall turun** (dokumen relevan terlewat).

Contoh ID: - **mahasiswa**, **kemahasiswaan**, **bermahasiswa** seharusnya jadi satu term **mahasiswa**; bila salah satu tidak ter-stem, query **mahasiswa** gagal menemukan dokumen yang menulis **kemahasiswaan**. - **pustaka** vs **perpustakaan** — bila **perpustakaan** tidak direduksi ke **pustaka**, dua dokumen bertopik sama tak dianggap berbagi term.

Kaitan dengan tugas kami: karena ranking VSM/TF-IDF dihitung pada ruang term yang sama, stemmer yang **overstem** akan menggelembungkan kemiripan palsu (cosine naik salah), sedangkan **understem** menurunkan cosine antara query dan dokumen yang sebenarnya relevan — keduanya menggeser peringkat. Algoritma Nazief-Adriani berbasis kamus relatif aman dari overstemming karena ada pengecekan kamus setelah tiap pengupasan imbuhan.

5. Analisis Metode Kelompok Lain di Drive

Tugas TF-IDF + Vector Space Model dikerjakan oleh **semua** kelompok, jadi yang dibandingkan di sini adalah **metode pembeda** tiap kelompok. Metode pembeda kelompok kami adalah **MinHash (Min-Wise Independent Permutations) + LSH**, sedangkan metode kelompok lain yang kami pelajari adalah **BM25 (Okapi BM25)**.

Perbedaan paling mendasar: keduanya menyelesaikan persoalan IR yang berbeda. - **MinHash** menjawab "seberapa mirip dokumen A dengan dokumen B" (dokumen-ke-dokumen, estimasi Jaccard similarity, simetris). Tujuan praktisnya deteksi near-duplicate, klastering, dan deduplikasi pada skala besar. - **BM25** menjawab "dokumen mana yang paling relevan terhadap query" (query-ke-dokumen, skor relevansi, asimetris). Tujuannya pemeringkatan hasil pencarian.

Jadi perbandingan ini bukan soal "mana lebih akurat pada tugas yang sama", melainkan kontras dua sub-masalah IR: **estimasi kemiripan himpunan** vs **pemeringkatan relevansi**.

Ringkas metode kami: MinHash + LSH

Tiap dokumen diubah menjadi himpunan **k-shingle** (kami pakai bigram, **K_SHINGLE = 2**). Untuk menghindari perbandingan himpunan besar secara langsung, dibangkitkan **K** fungsi hash universal lalu diambil nilai hash minimum tiap dokumen, menghasilkan **signature matrix** berukuran **K x N**. Jaccard similarity diestimasi dari proporsi posisi signature yang cocok antar dua dokumen. Akurasi estimasi naik seiring **K** dengan error berbanding terbalik terhadap akar **K** (pola diminishing returns).

Metode kelompok lain: BM25

BM25 adalah model **probabilistik** pemeringkatan. Rumus inti:

$$BM25(q, d) = \sum_{t \in q} IDF(t) \cdot \frac{f_{t,d}(k_1 + 1)}{f_{t,d} + k_1 \left(1 - b + b \frac{|d|}{avgdl}\right)}$$

dengan k_1 (saturasi TF, umum 1.2 sampai 2.0) dan b (normalisasi panjang, umum 0.75).

Perbedaan dengan metode kami (MinHash + LSH)

Aspek	MinHash + LSH (kelompok kami)	BM25 (kelompok lain)
Tujuan utama	Estimasi kemiripan antar-dokumen (Jaccard), deteksi duplikat	Pemeringkatan relevansi dokumen terhadap query
Arah perbandingan	Dokumen vs dokumen (simetris)	Query vs dokumen (asimetris)
Representasi dokumen	Himpunan k-shingle lalu signature ringkas	Bag-of-words berbobot (TF, IDF, panjang)
Dasar teori	Probabilistik hashing (Min-Wise Independent Permutations)	Probabilistic Relevance Framework
Frekuensi term	Diabaikan (hanya ada/tidaknya shingle, biner)	Diperhitungkan dengan saturasi via k_1
Sifat hasil	Estimasi (aproksimasi), ada error sekitar $1/akar(K)$	Skor eksak deterministik per pasangan query-dokumen
Parameter	K (jumlah fungsi hash), k (ukuran shingle)	k_1 (saturasi TF), b (normalisasi panjang)
Kekuatan skala	Sangat hemat memori, kandidat duplikat cepat via LSH untuk korpus besar	Cepat untuk ranking per query, tidak dirancang untuk all-pairs similarity

Keuntungan BM25 (dibanding MinHash untuk tugas pemeringkatan)

- **Menghasilkan peringkat relevansi langsung** terhadap query, cocok untuk mesin pencari; MinHash tidak memberi skor relevansi query-dokumen.
- **Membobot term** lewat TF (dengan saturasi) dan IDF, sedangkan MinHash hanya melihat keanggotaan himpunan shingle (ada atau tidak).
- **Hasil deterministik dan eksak**, tanpa galat estimasi seperti MinHash.

Kelemahan BM25 (dibanding MinHash)

- **Tidak dirancang untuk perbandingan all-pairs** antar dokumen; menghitung kemiripan semua pasangan menjadi mahal (sekitar $O(n^2)$) tanpa trik seperti LSH yang dimiliki MinHash.
- **Perlu tuning k_1 dan b** ; nilai default tak selalu optimal untuk korpus tertentu (mis. jurnal pendek Bahasa Indonesia).
- **Tetap bag-of-words**, tidak menangani sinonim/polisemi. *Beruang* vs *uang* tetap masalah; *mobil* vs *kendaraan* tetap dianggap beda.

Keuntungan & kelemahan metode kami sendiri (MinHash)

- **Keunggulan:** kompresi besar (signature berisi K bilangan menggantikan himpunan shingle yang besar), estimasi Jaccard cepat, dan dengan LSH menemukan kandidat duplikat dalam waktu mendekati linear sehingga skalabel untuk korpus sangat besar.
- **Kelemahan:** hasilnya hanya estimasi (akurasi bergantung K , error sekitar $1/\sqrt{K}$, diminishing returns), berbasis kemiripan himpunan sehingga tidak mengukur relevansi terhadap query dan tidak membobot term penting, serta sensitif pada pemilihan ukuran shingle k .